LAPORAN

WORKSHOP Mobile Web Server

(TIF310808)

Acara 9

SEMESTER 3



Nama Tugas

OOP MVC LANJUTAN:
OBJECT, DEPENDENCY, GETTER SETTER

Nama:

Adam Ridho Saifulloh

NIM:

E41250216

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Pengembangan aplikasi web yang semakin kompleks membutuhkan struktur kode yang terorganisasi agar mudah dipelihara, diuji, dan dikembangkan. Salah satu pendekatan yang digunakan adalah Object-Oriented Programming (OOP) yang memanfaatkan class, object, encapsulation, dan relasi antarobjek. Dalam arsitektur MVC, pemisahan tanggung jawab antara Model, View, dan Controller membantu menjaga agar setiap bagian aplikasi memiliki fungsi yang jelas.

Pada Acara 9, praktik difokuskan pada OOP MVC lanjutan dengan penerapan Object Composition, Dependency Injection sederhana, serta Getter dan Setter. Pada project yang dikerjakan, koneksi database dikelola oleh class Database, kemudian objek Database diberikan kepada MahasiswaRepository melalui constructor. Repository tersebut selanjutnya diberikan kepada MahasiswaController. Pendekatan ini membuat ketergantungan antarbagian program lebih jelas dan mengurangi pembuatan koneksi database secara langsung di Controller.

1.2 Rumusan Masalah

- Bagaimana menerapkan Object Composition pada aplikasi MVC?
- Bagaimana menerapkan Constructor Dependency Injection pada Controller dan Repository?
- Bagaimana menggunakan Getter dan Setter untuk mengontrol akses terhadap atribut object?
- Bagaimana menerapkan validasi data melalui Setter?
- Bagaimana memastikan fitur tambah, tampil, ubah, hapus, dan pencarian data mahasiswa tetap berjalan setelah refactoring?

1.3 Tujuan Praktikum

- Memahami penerapan prinsip OOP dalam pengembangan backend web.
- Menerapkan Object Composition pada hubungan Database, Repository, dan Controller.
- Menerapkan Constructor Dependency Injection.
- Menerapkan encapsulation menggunakan atribut private.
- Membuat Getter dan Setter beserta validasi sederhana.

- Memastikan proses CRUD data mahasiswa tetap berjalan setelah refactoring.

BAB II
DASAR TEORI

2.1 Object Composition

Object Composition merupakan pendekatan ketika sebuah object memiliki atau menggunakan object lain untuk menjalankan tugas tertentu. Pada project ini, MahasiswaController memiliki object MahasiswaRepository, sedangkan MahasiswaRepository menggunakan object Database untuk memperoleh koneksi PDO.

2.2 Dependency Injection

Dependency Injection adalah teknik memberikan dependency dari luar class, bukan membuat dependency tersebut langsung di dalam class. Pada project ini, MahasiswaRepository menerima object Database melalui constructor. MahasiswaController juga menerima object MahasiswaRepository melalui constructor.

```
class MahasiswaController
{
    private MahasiswaRepository $repo;

    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }
}
```

2.3 Encapsulation

Encapsulation digunakan untuk membatasi akses langsung terhadap data internal object. Pada class Mahasiswa, atribut seperti id, nim, nama, email, prodiId, dan angkatan dibuat private. Data tersebut diakses melalui method Getter dan diubah melalui Setter.

2.4 Getter dan Setter

Getter digunakan untuk mengambil nilai atribut, sedangkan Setter digunakan untuk mengubah nilai atribut. Setter juga dapat menjadi tempat validasi agar data yang masuk memenuhi aturan yang ditentukan.

```
public function setNim(string $nim): void
{
    $nim = trim($nim);
```

```
    if ($nim === '') {
        throw new InvalidArgumentException('NIM wajib diisi.');
    }
    if (!ctype_digit($nim)) {
        throw new InvalidArgumentException('NIM harus berupa angka.');
    }

    $this->nim = $nim;
}
```

2.5 MVC pada Project

Model menangani representasi data mahasiswa, Repository menangani operasi database, Controller menangani alur request dan proses aplikasi, sedangkan View menampilkan data kepada pengguna. Dengan pembagian tersebut, perubahan pada satu bagian tidak harus mengubah seluruh bagian aplikasi.

BAB III

ALAT DAN BAHAN

3.1 Perangkat yang Digunakan

- Laptop/PC.
- Visual Studio Code.
- XAMPP/Laragon.
- PHP.
- MySQL/MariaDB dan phpMyAdmin.
- Web Browser.
- HTML, CSS, dan Bootstrap.
- Modul BKPM Workshop Sistem Informasi Web Server.

3.2 Struktur Project

```
Bkpm_Acara_9/
├── app/
│   ├── Controllers/
│   │   ├── AuthController.php
│   │   └── MahasiswaController.php
│   ├── Models/
│   │   └── Mahasiswa.php
│   ├── Repositories/
│   │   └── MahasiswaRepository.php
│   ├── Database.php
│   ├── helpers.php
│   └── Views/
│       ├── auth/
│       ├── mahasiswa/
│       ├── header.php
│       └── footer.php
├── config/
│   └── database.php
├── database/
│   └── bkpm_acara_7 (1).sql
└── public/
    └── index.php
```

BAB IV
PROSEDUR PRAKTIKUM

4.1 Persiapan Project

1. Mempersiapkan XAMPP dan memastikan Apache serta MySQL dapat digunakan.
2. Membuka project Bkpm_Acara_9 pada Visual Studio Code.
3. Memastikan database pada file konfigurasi mengarah ke database bkpm_acara_7.
4. Memastikan struktur folder aplikasi sesuai dengan pola MVC yang digunakan.

4.2 Implementasi Class Database

Class Database bertugas mengelola koneksi PDO. Property $pdo dibuat private dan koneksi dibuat ketika getConnection() pertama kali dipanggil. Konfigurasi host, database, username, password, dan charset dibaca dari config/database.php.

4.3 Implementasi MahasiswaRepository

MahasiswaRepository menerima object Database melalui constructor. Object tersebut digunakan untuk memperoleh koneksi PDO. Repository menyediakan operasi all(), find(), create(), update(), delete(), dan daftarProdi(). Query menggunakan prepared statement.

```php
<?php

require_once __DIR__ . '/../Database.php';
require_once __DIR__ . '/../Models/Mahasiswa.php';

class MahasiswaRepository
{
    private PDO $pdo;

    // Constructor injection: Database diberikan dari luar, tidak dibuat di
sini
    public function __construct(Database $db)
    {
        $this->pdo = $db->getConnection();
    }

    // Semua mahasiswa + nama prodi (JOIN). Jika $keyword diisi, cari nama atau
NIM dengan LIKE
    public function all(string $keyword = ''): array
    {
```

```
        $sql = "SELECT m.*, p.nama AS prodi_nama
                FROM mahasiswa m
                JOIN prodi p ON m.prodi_id = p.id";
        $params = [];

        if ($keyword !== '') {
            $sql .= " WHERE m.nama LIKE :nama OR m.nim LIKE :nim";
            $params['nama'] = '%' . $keyword . '%';
            $params['nim']  = '%' . $keyword . '%';
        }

        $sql .= " ORDER BY m.nim";

        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);

        $hasil = [];
        foreach ($stmt->fetchAll() as $row) {
            $hasil[] = Mahasiswa::dariBaris($row);
        }
        return $hasil;
    }

    public function find(int $id): ?Mahasiswa
    {
        $stmt = $this->pdo->prepare("SELECT * FROM mahasiswa WHERE id = :id");
        $stmt->execute(['id' => $id]);
        $row = $stmt->fetch();

        return $row ? Mahasiswa::dariBaris($row) : null;
    }

    public function create(Mahasiswa $m): void
    {
        $stmt = $this->pdo->prepare(
            "INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan)
             VALUES (:nim, :nama, :email, :prodi_id, :angkatan)"
        );
        $stmt->execute([
            'nim'      => $m->getNim(),
            'nama'     => $m->getNama(),
            'email'    => $m->getEmail(),
            'prodi_id' => $m->getProdiId(),
            'angkatan' => $m->getAngkatan(),
        ]);
```

```
    }

    public function update(Mahasiswa $m): void
    {
        $stmt = $this->pdo->prepare(
            "UPDATE mahasiswa
             SET nim = :nim, nama = :nama, email = :email, prodi_id =
:prodi_id, angkatan = :angkatan
             WHERE id = :id"
        );
        $stmt->execute([
            'nim'      => $m->getNim(),
            'nama'     => $m->getNama(),
            'email'    => $m->getEmail(),
            'prodi_id' => $m->getProdiId(),
            'angkatan' => $m->getAngkatan(),
            'id'       => $m->getId(),
        ]);
    }

    public function delete(int $id): void
    {
        $stmt = $this->pdo->prepare("DELETE FROM mahasiswa WHERE id = :id");
        $stmt->execute(['id' => $id]);
    }

    // Daftar prodi untuk pilihan di form
    public function daftarProdi(): array
    {
        return $this->pdo->query("SELECT id, kode, nama FROM prodi ORDER BY
kode")->fetchAll();
    }
}
```

4.4 Implementasi MahasiswaController

MahasiswaController menerima MahasiswaRepository melalui constructor. Controller kemudian menggunakan repository untuk menjalankan proses daftar mahasiswa, tambah, edit, update, hapus, dan pencarian.

```
<?php

require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';
```

```php
class MahasiswaController
{
    private MahasiswaRepository $repo;

    // Constructor injection: Repository diberikan dari luar (lihat
public/index.php)
    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }

    public function index(): void
    {
        $keyword = trim($_GET['q'] ?? '');
        $daftar  = $this->repo->all($keyword);
        require __DIR__ . '/../Views/mahasiswa/index.php';
    }

    public function create(): void
    {
        $this->tampilForm(new Mahasiswa(), url('/mahasiswa/store'));
    }

    public function store(): void
    {
        [$m, $errors] = $this->ambilInput();

        if (!$errors) {
            try {
                $this->repo->create($m);
                redirect('/mahasiswa', 'Data mahasiswa berhasil ditambahkan.');
            } catch (PDOException $e) {
                $errors = $this->errorDatabase($e);
            }
        }

        $this->tampilForm($m, url('/mahasiswa/store'), $errors, $_POST);
    }

    public function edit(): void
    {
        $m = $this->repo->find((int) ($_GET['id'] ?? 0));
        if ($m === null) {
```

```
            redirect('/mahasiswa', 'Data mahasiswa tidak ditemukan.',
'danger');
        }

        $this->tampilForm($m, url('/mahasiswa/update'));
    }

    public function update(): void
    {
        $id = (int) ($_POST['id'] ?? 0);
        if ($this->repo->find($id) === null) {
            redirect('/mahasiswa', 'Data mahasiswa tidak ditemukan.',
'danger');
        }

        [$m, $errors] = $this->ambilInput($id);

        if (!$errors) {
            try {
                $this->repo->update($m);
                redirect('/mahasiswa', 'Data mahasiswa berhasil diperbarui.');
            } catch (PDOException $e) {
                $errors = $this->errorDatabase($e);
            }
        }

        $this->tampilForm($m, url('/mahasiswa/update'), $errors, $_POST);
    }

    public function destroy(): void
    {
        $this->repo->delete((int) ($_POST['id'] ?? 0));
        redirect('/mahasiswa', 'Data mahasiswa berhasil dihapus.');
    }

    // Isi objek Mahasiswa lewat setter. Setter yang gagal validasi
menghasilkan pesan error.
    private function ambilInput(int $id = 0): array
    {
        $m = new Mahasiswa();
        $m->setId($id);
        $errors = [];

        $input = [
            'nim'      => ['setNim',      $_POST['nim'] ?? ''],
```

```
            'nama'     => ['setNama',     $_POST['nama'] ?? ''],
            'email'    => ['setEmail',    $_POST['email'] ?? ''],
            'prodi_id' => ['setProdiId',  (int) ($_POST['prodi_id'] ?? 0)],
            'angkatan' => ['setAngkatan', (int) ($_POST['angkatan'] ?? 0)],
        ];

        foreach ($input as $field => [$setter, $nilai]) {
            try {
                $m->$setter($nilai);
            } catch (InvalidArgumentException $e) {
                $errors[$field] = $e->getMessage();
            }
        }

        return [$m, $errors];
    }

    // NIM sudah dipakai (kolom UNIQUE) -> error 23000
    private function errorDatabase(PDOException $e): array
    {
        if ($e->getCode() === '23000') {
            return ['nim' => 'NIM sudah terdaftar.'];
        }
        throw $e;
    }

    private function tampilForm(Mahasiswa $m, string $action, array $errors =
[], array $old = []): void
    {
        $prodiList = $this->repo->daftarProdi();
        require __DIR__ . '/../Views/mahasiswa/form.php';
    }
}
```

4.5 Implementasi Getter dan Setter

Atribut pada class Mahasiswa dibuat private. Getter digunakan untuk mengambil nilai, sedangkan Setter digunakan untuk memasukkan data sekaligus melakukan validasi. Validasi yang diterapkan antara lain NIM wajib diisi dan berupa angka, nama tidak boleh kosong, email harus valid jika diisi, program studi harus dipilih, dan angkatan harus berada pada rentang yang valid.

```php
<?php

class Mahasiswa
{
    // Semua atribut private: hanya bisa diakses lewat getter dan setter
    private int $id = 0;
    private string $nim = '';
    private string $nama = '';
    private string $email = '';
    private int $prodiId = 0;
    private int $angkatan;
    private string $prodiNama = '';   // hanya untuk tampilan (hasil JOIN)

    public function __construct()
    {
        $this->angkatan = (int) date('Y');
    }
// ---------- Getter ----------
    public function getId(): int          { return $this->id; }
    public function getNim(): string      { return $this->nim; }
    public function getNama(): string     { return $this->nama; }
    public function getEmail(): string    { return $this->email; }
    public function getProdiId(): int     { return $this->prodiId; }
    public function getAngkatan(): int    { return $this->angkatan; }
    public function getProdiNama(): string { return $this->prodiNama; }

    // ---------- Setter (dengan validasi sederhana) ----------
    public function setId(int $id): void
    {
        $this->id = $id;
    }

    public function setNim(string $nim): void
    {
        $nim = trim($nim);

        if ($nim === '') {
            throw new InvalidArgumentException('NIM wajib diisi.');
        }
        if (!ctype_digit($nim)) {
            throw new InvalidArgumentException('NIM harus berupa angka.');
        }
        if (strlen($nim) > 20) {
            throw new InvalidArgumentException('NIM maksimal 20 digit.');
        }
```

```
        $this->nim = $nim;
    }

    public function setNama(string $nama): void
    {
        $nama = trim($nama);

        if ($nama === '') {
            throw new InvalidArgumentException('Nama tidak boleh kosong.');
        }
        if (strlen($nama) > 100) {
            throw new InvalidArgumentException('Nama maksimal 100 karakter.');
        }

        $this->nama = $nama;
    }

    public function setEmail(string $email): void
    {
        $email = trim($email);

        if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException('Format email tidak valid.');
        }

        $this->email = $email;
    }

    public function setProdiId(int $prodiId): void
    {
        if ($prodiId <= 0) {
            throw new InvalidArgumentException('Pilih program studi.');
        }

        $this->prodiId = $prodiId;
    }

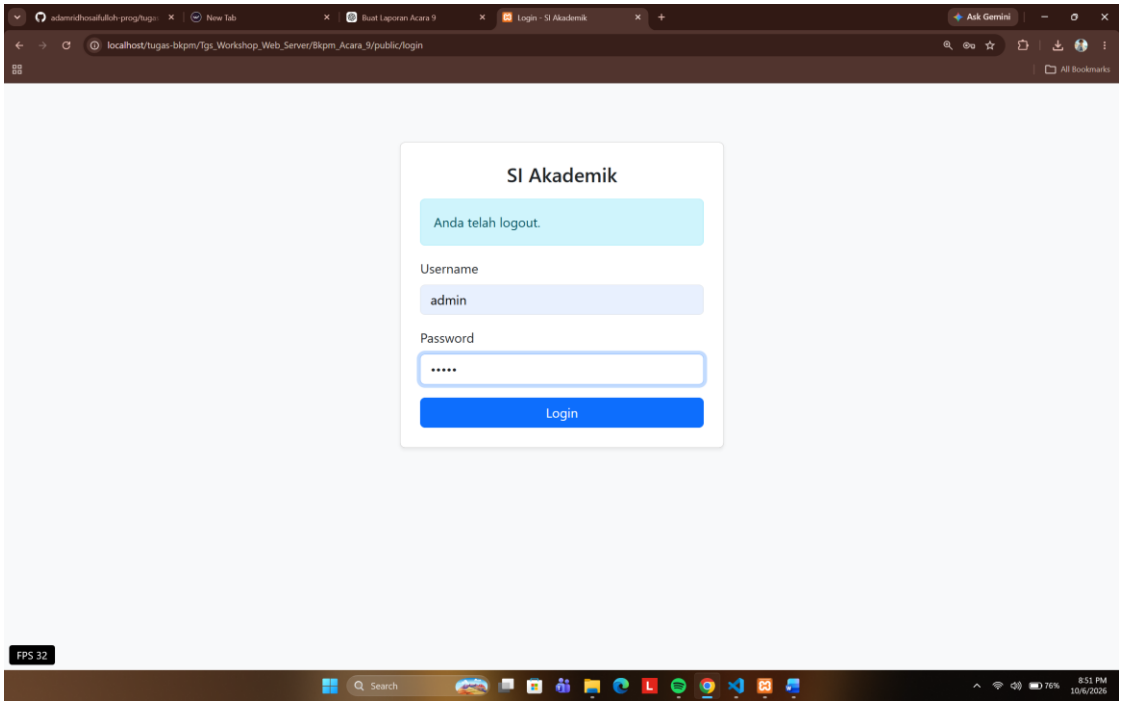
    public function setAngkatan(int $angkatan): void
    {
        if ($angkatan < 1990 || $angkatan > (int) date('Y') + 1) {
            throw new InvalidArgumentException('Angkatan tidak valid.');
        }

        $this->angkatan = $angkatan;
```

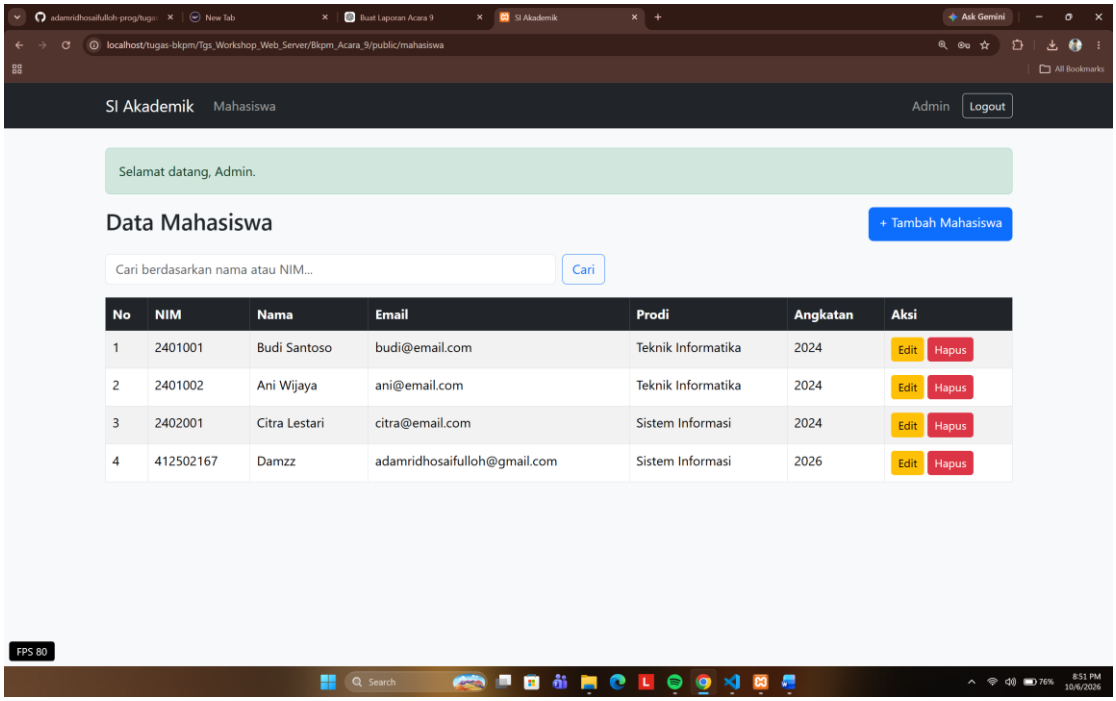
```
    }
}
```

4.6 Pengujian Fitur

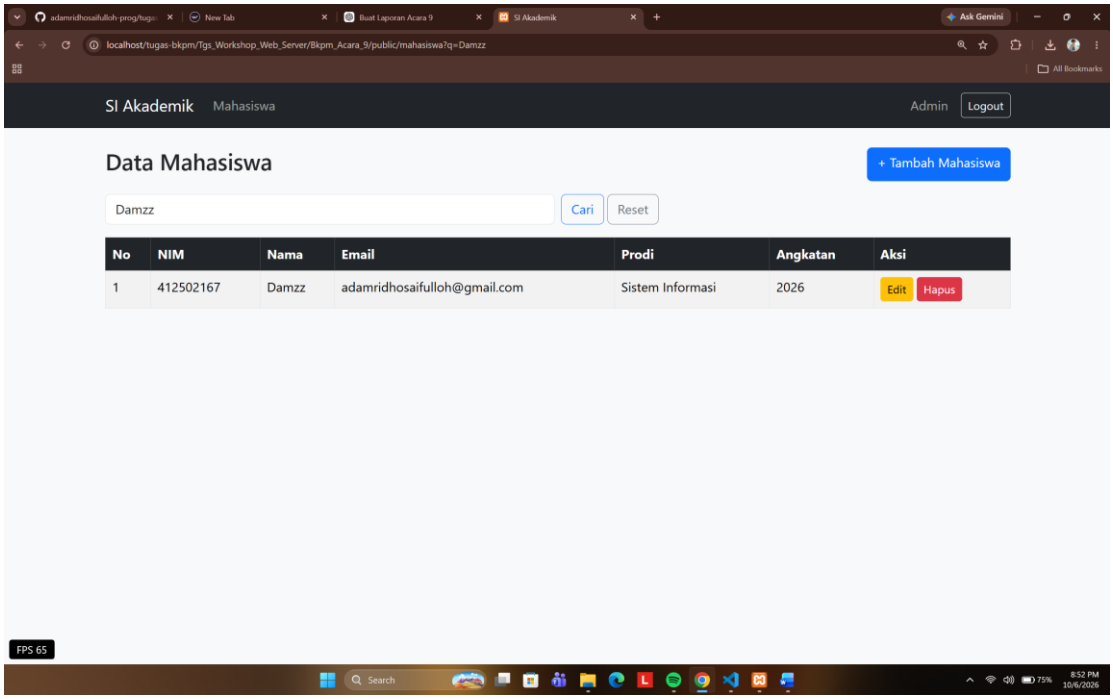
5. Login menggunakan akun admin yang tersedia pada AuthController.



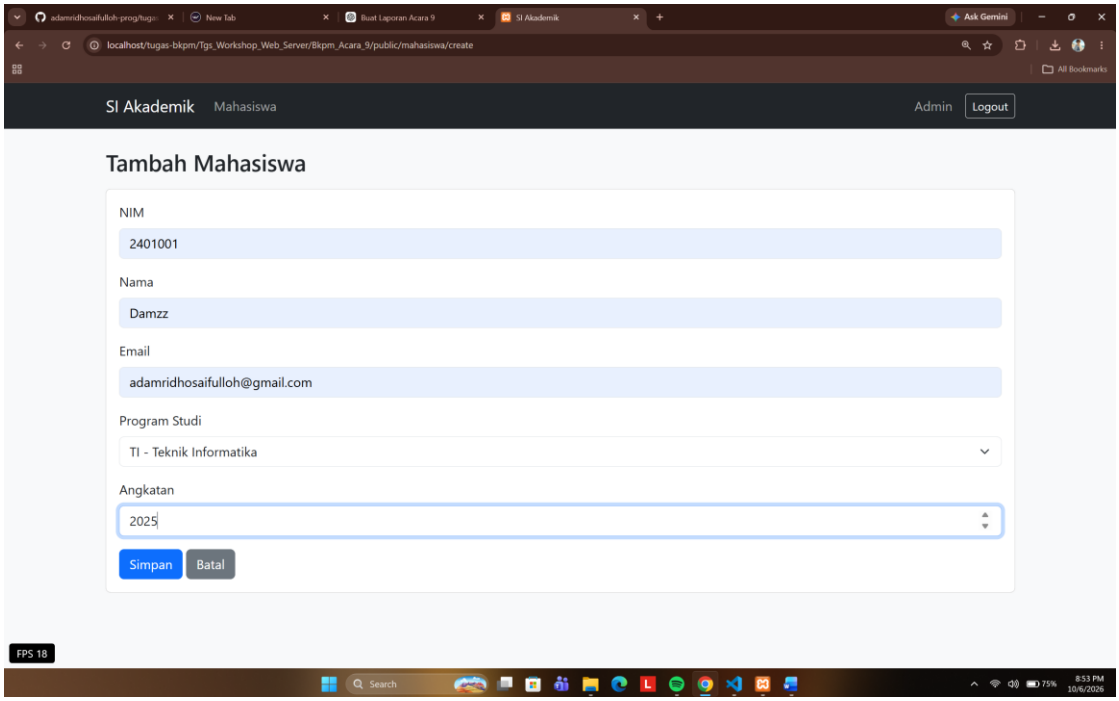
6. Membuka halaman Data Mahasiswa.



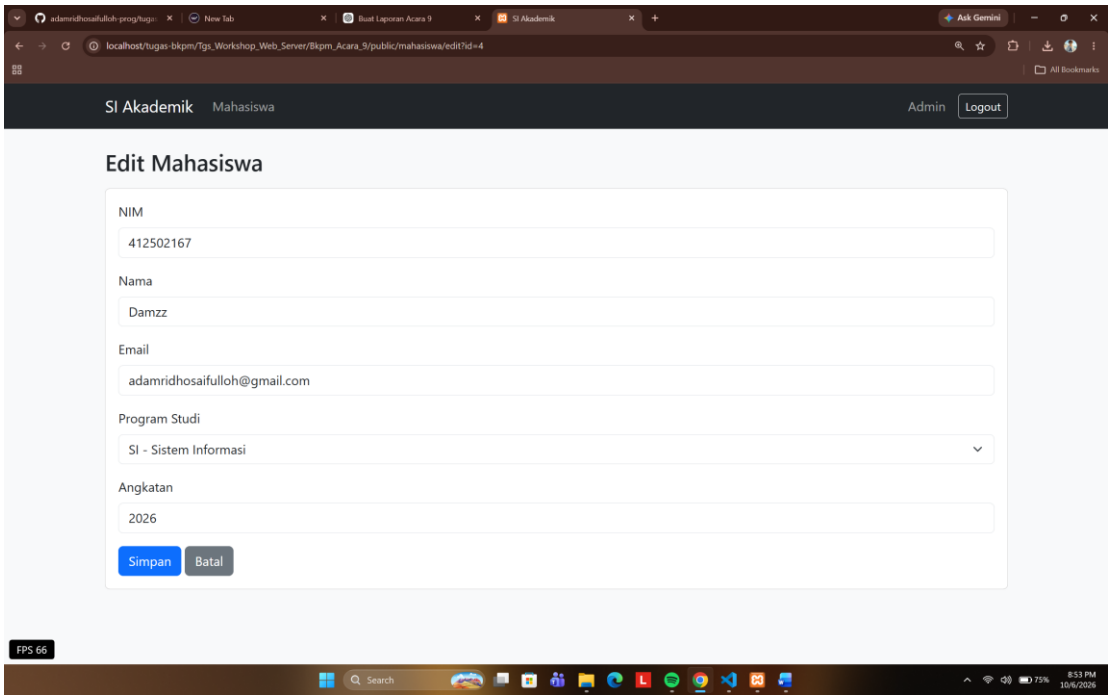
7. Mencari data berdasarkan nama atau NIM.



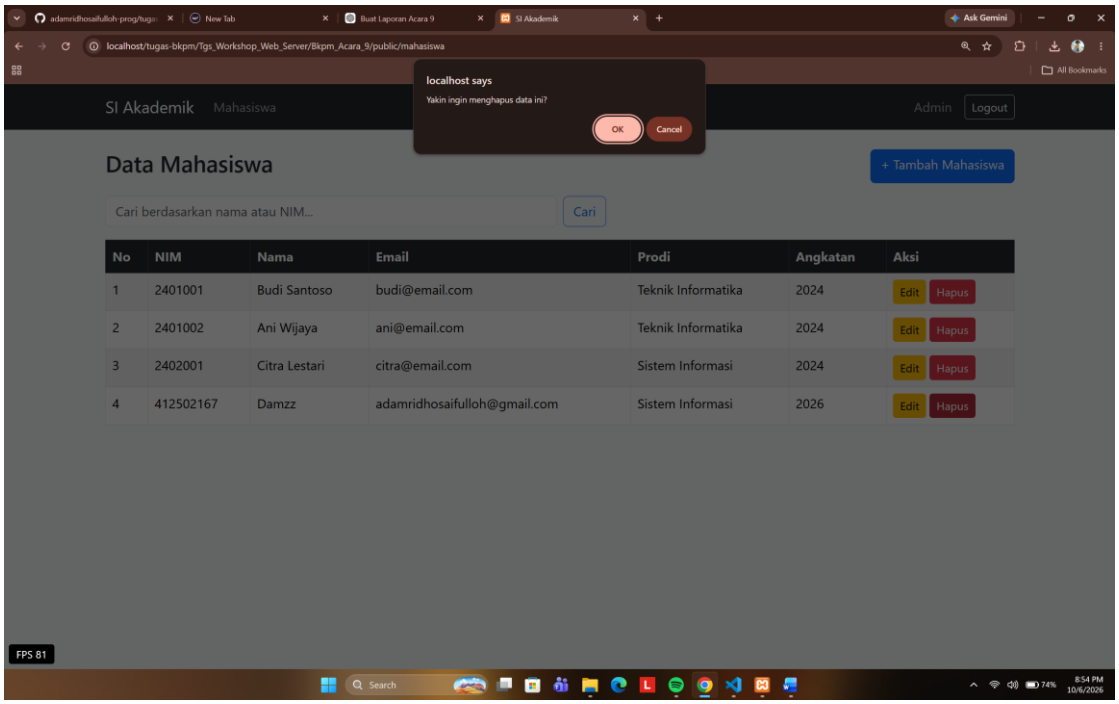
8. Menambahkan data mahasiswa.



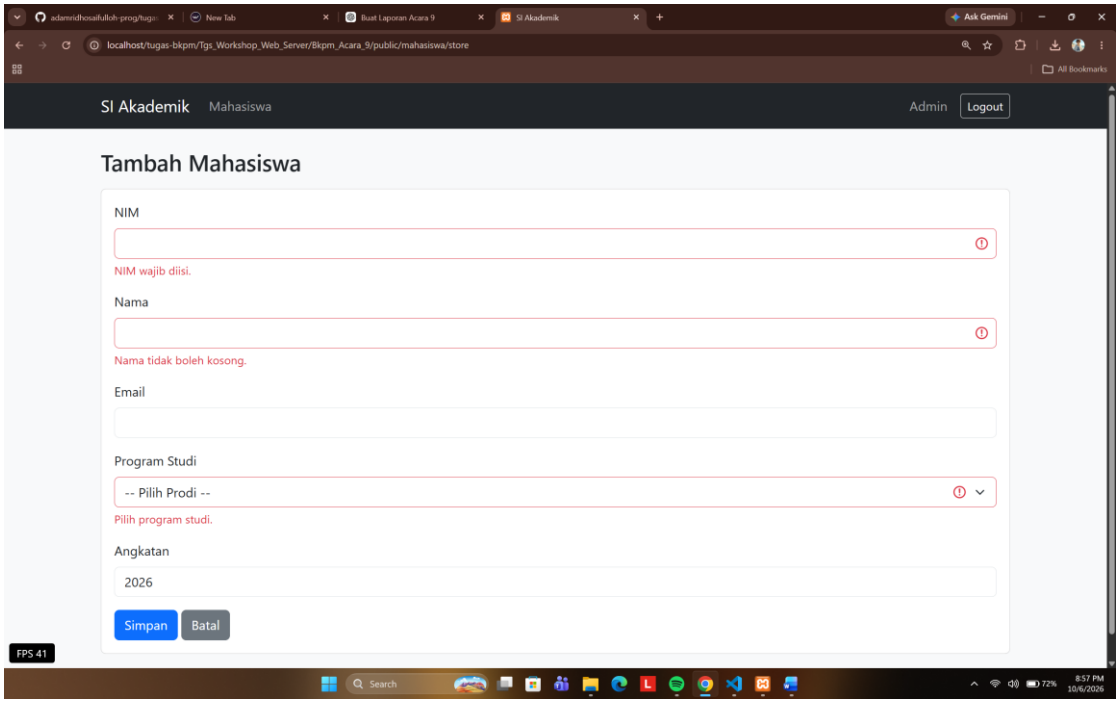
9. Mengubah data mahasiswa.



10. Menghapus data mahasiswa.



11. Menguji validasi input melalui Setter.



BAB V HASIL DAN PEMBAHASAN

5.1 Hasil Implementasi

Berdasarkan source code project Bkpm_Acara_9 yang digunakan dalam praktikum, implementasi OOP MVC lanjutan telah diterapkan. Alur dependency pada aplikasi adalah sebagai berikut:

```
Database
  ↓
MahasiswaRepository
  ↓
MahasiswaController
  ↓
View Mahasiswa
```

Class Database menyediakan koneksi PDO. MahasiswaRepository menerima Database sebagai dependency dan menjalankan operasi database. MahasiswaController menerima Repository sebagai dependency sehingga Controller tidak membuat koneksi database secara langsung.

5.2 Implementasi Object Composition

Object Composition terlihat pada hubungan antarobject. MahasiswaController menggunakan MahasiswaRepository, sedangkan MahasiswaRepository menggunakan Database. Dengan cara ini, setiap class memiliki tanggung jawab yang lebih terarah.

5.3 Implementasi Getter dan Setter

Pada class Mahasiswa terdapat Getter untuk id, NIM, nama, email, program studi, angkatan, dan nama program studi. Setter digunakan untuk id, NIM, nama, email, program studi, dan angkatan. Setter tidak hanya mengubah nilai tetapi juga melakukan validasi.

5.4 Implementasi Validasi

Field	Validasi	Hasil Jika Tidak Valid
NIM	Tidak boleh kosong, harus angka, maksimal 20 digit	InvalidArgumentException
Nama	Tidak boleh kosong, maksimal 100 karakter	InvalidArgumentException
Email	Jika diisi harus memiliki format email valid	InvalidArgumentException

Program Studi	Harus memilih prodi	InvalidArgumentException
Angkatan	Harus berada pada rentang tahun yang valid	InvalidArgumentException

5.5 Implementasi CRUD dan Pencarian

Repository menyediakan method create(), update(), dan delete() untuk operasi data. Method all() mengambil data mahasiswa dan nama program studi menggunakan JOIN. Method all() juga menerima keyword pencarian dan menerapkan LIKE pada nama atau NIM dengan prepared statement.

```
WHERE m.nama LIKE :nama OR m.nim LIKE :nim

$params['nama'] = '%' . $keyword . '%';
$params['nim']  = '%' . $keyword . '%';
```

5.6 Analisis Hasil

Penerapan dependency injection membuat hubungan antarclass lebih terstruktur karena MahasiswaController tidak perlu mengetahui bagaimana koneksi database dibuat. Selain itu, penggunaan private property pada Mahasiswa memperkuat encapsulation. Validasi pada Setter membantu mencegah data yang tidak sesuai masuk ke object. Dengan refactoring tersebut, fungsi CRUD tetap ditempatkan pada Repository sementara Controller berfokus pada pengaturan alur aplikasi.

BAB VI KESIMPULAN

Praktikum Acara 9 berhasil menerapkan konsep OOP MVC lanjutan pada aplikasi manajemen data mahasiswa. Object Composition diterapkan melalui hubungan Database, MahasiswaRepository, dan MahasiswaController. Constructor Dependency Injection digunakan agar dependency diberikan dari luar class sehingga kode lebih fleksibel dan mudah diuji.

Class Mahasiswa menerapkan encapsulation dengan menjadikan atribut private serta menyediakan Getter dan Setter. Setter dilengkapi validasi sederhana untuk NIM, nama, email, program studi, dan angkatan. Selain itu, fungsi CRUD dan pencarian mahasiswa tetap dapat dijalankan melalui Repository. Dengan penerapan tersebut, struktur aplikasi menjadi lebih rapi, terpisah, dan lebih mudah dikembangkan.